

แนวทางของ Gemini (พัฒนา PPO_V2)

การจะพัฒนาโมเดลจาก PPO_V2 ให้สามารถสร้าง Alpha ที่แท้จริง ($Edge > 0$), ดัน Sharpe Ratio ให้อยู่ในระดับ 1.5 – 2.0+ และควบคุม Max Drawdown ให้ต่ำที่สุด เพื่อขึ้น Production ได้นั้น ไม่สามารถทำได้เพียงแค่การจูน Hyperparameters (เช่น ปรับ Learning Rate หรือ Batch Size)

ในทาง Quantitative Finance หากพอร์ตโฟลิโอมีเพียง Equity Long-Only (DIA, QQQ, SPY) + Cash ตลาดหุ้นสหรัฐฯ มี Sharpe Ratio ระยะยาวตามธรรมชาติอยู่ที่เพียง ~0.5 – 0.7 การจะดัน Sharpe ทะลุ 1.5+ ด้วยสินทรัพย์ที่เคลื่อนไหวไปในทิศทางเดียวกัน (High Correlation) โดยถือเงินสดเป็นหลัก เป็นไปไม่ได้ในเชิงคณิตศาสตร์ (Mathematical Impossibility) เพราะปีที่ตลาดพัง (เช่น 2008 หรือ 2022) ทุกตัวลงพร้อมกัน และปีที่ตลาดกระชาก การถือเงินสดจะดึงผลตอบแทนลงทันที

นี่คือ พิมพ์เขียวเชิงสถาปัตยกรรม (Production-Ready Quantitative Architecture) ที่ต้องปรับปรุง V2 ทั้งระบบ:

เสาหลักที่ 1: สถาปัตยกรรมแบบ 2 ชั้น (Two-Tier Hierarchical Architecture)

ข้อผิดพลาดสำคัญของ V2 คือการบังคับให้ RL Agent ทำ 2 หน้าที่พร้อมกัน: (1) เลือกว่าจะซื้ออะไร (Asset Allocation) และ (2) ควบคุมความเสี่ยงไม่ให้พอร์ตพัง (Risk Management) ส่งผลให้ Agent เลือกทางที่ปลอดภัยที่สุดคือการ "หนีไปนอนถือเงินสด (Cash Trap)"

ระบบระดับ Production ของกองทุน Quant ระดับโลก (เช่น AQR, Two Sigma, Bridgewater) จะแยกสองหน้าที่นี้ออกจากกันอย่างเด็ดขาด:

[Market Features & Macro Data]



Tier 1: RL Agent (Alpha)	← โฟกัสเฉพาะ "Relative Strength" เท่านั้น
ตัดสินใจสัดส่วนเฉพาะสินทรัพย์	(บังคับ Fully Invested: DIA vs QQQ vs SPY)

Raw Target Weights (Sum = 1.0)



Tier 2: Volatility Targeting	← Deterministic Risk Engine (ไม่ใช่ RL)
(Systematic Risk Overlay)	ปรับ Gross Exposure ตามความผันผวนของตลาด

Executed Weights + Tactical Cash



[Execution at Open(t+1)]

กลไก Volatility Targeting (กด Max Drawdown แบบอัตโนมัติ)

แทนที่จะปล่อยให้โมเดลเดาว่าจะถือเงินสดเท่าไร ให้ใช้สูตร **Target Volatility Engine** (เช่น ตั้งเป้าความผันผวนพอร์ตที่ $\sigma_{\text{target}} = 10\%$ ต่อปี):

$$\text{Scalar}_t = \min\left(1.0, \frac{\sigma_{\text{target}}}{\hat{\sigma}_{\text{realized}, t}}\right)$$

- **ช่วงตลาดนิ่ง/กระทิง (Low Vol):** $\hat{\sigma}$ ต่ำ $\rightarrow \text{Scalar} \approx 1.0$ (ลงทุนเต็ม 100% กวาด Alpha เต็มเม็ดเต็มหน่วย ไม่ตกรถ)
- **ช่วงวิกฤต/ตลาดผันผวนสูง (High Vol เช่น COVID, 2022):** $\hat{\sigma}$ พุ่งสูง $\rightarrow \text{Scalar}$ จะหดตัวลงเหลือ 0.3 - 0.5 อัตโนมัติ (เงินส่วนที่เหลือจะกลายเป็น Cash กันทีเดียวโดยไม่ต้องพึ่งพาการตัดสินใจของ RL)
- **ผลลัพธ์:** Drawdown จะถูกตัดตอนทันทีก่อนที่จะลุกลาม และ Sharpe Ratio จะเพิ่มขึ้นทันที 30-50% จากคุณสมบัติ Volatility Clustering

เสาหลักที่ 2: ปฏิรูป Reward Function แก้ปัญหา Cash Trap

ใน V2 Reward ใช้ Log Return ลบด้วย Drawdown Penalty แบบคงที่ ทำให้ Agent หวาดกลัวตลาดขาลงจนยอมเสียโอกาสในขาขึ้นทั้งหมด ต้องเปลี่ยน Reward ให้มุ่งเน้น **Alpha** และ **Risk-Adjusted Return** โดยตรง:

1. Benchmark-Relative Excess Return (Alpha Reward)

บังคับให้ Agent ต้องวัดผลเทียบกับ Benchmark (Equal-Weight หรือ SPY) ก้าวต่อก้าว:

$$R_{\alpha, t} = r_{\text{agent}, t} - r_{\text{benchmark}, t}$$

- ถ้าตลาดบวก 2% แต่ Agent ถือเงินสดได้ 0% $\rightarrow$ Reward ติดลบ -2% ทันที (Agent จะโดนลงโทษหนักมากหาก "ตกรถ")
- ถ้าตลาดลบ -3% แต่ Agent เสียเพียง -0.5% $\rightarrow$ Reward เป็นบวก +2.5% (Agent เรียนรู้ว่าการรักษาเงินสดในขาลงคือการชนะ)

2. Differential Sharpe Ratio (Moody & Saffell)

เปลี่ยนเป้าหมายของ Policy Gradient จากการเพิ่ม Return สะสม ไปสู่การ **Maximize Sharpe Ratio แบบ Online Recursive**:

$$D_t = \frac{B_{t-1} \Delta A_t - \frac{1}{2} A_{t-1} \Delta B_t}{(B_{t-1} - A_{t-1})^2}$$

โดยที่ A_t คือค่าเฉลี่ยผลตอบแทนถ่วงน้ำหนักแบบ Exponential ($\text{EMA}(r_t)$) และ B_t คือค่าเฉลี่ยกำลังสองของผลตอบแทน ($\text{EMA}(r_t^2)$) การใช้ Reward รูปแบบนี้ได้รับการพิสูจน์แล้วว่าช่วยผลักดันให้พอร์ตสร้าง Sharpe เกิน 1.5 ได้ดีที่สุดในทางสถิติ

3. Downside Semi-Variance Penalty (Sortino Objective)

ห้ามลงโทษความผันผวนฝั่งขาขึ้น (Upside Volatility) ให้ลงโทษเฉพาะส่วนที่พอร์ตติดลบเท่านั้น:

$$\text{Penalty}_t = \lambda \cdot \min(0, r_{\text{agent}, t})^2$$

เสาหลักที่ 3: ขยาย Asset Universe เพื่อดึง Uncorrelated Alpha

การใช้เพียง DIA, QQQ, SPY มีข้อจำกัดทางคณิตศาสตร์ เพราะค่า Correlation ระหว่างกันสูงถึง \$0.85 - 0.95\$ (ความหลากหลายของพอร์ตต่ำมาก) หากต้องการ Sharpe ทะลุ 1.5 – 2.0 ในทุกสภาวะตลาด จำเป็นต้องเพิ่ม สินทรัพย์ที่วิ่งสวนทางหรือไม่มีความสัมพันธ์กัน (Flight-to-Safety Assets):

1. **US Treasuries (พันธบัตรรัฐบาลสหรัฐฯ):** เพิ่ม TLT (20+ Year Treasury) หรือ IEF (7-10 Year) — ในยามที่ตลาดหุ้นเกิด Liquidity Shock หรือ Recession เงินทุนสถาบันจะไหลเข้าพันธบัตร ทำให้ Agent มีทางเลือกในการทำกำไรฝั่ง Long โดยไม่ต้องถือเงินสดเปล่า ๆ
2. **Inflation / Alternative Asset:** เพิ่ม GLD (Gold) — ในสภาวะ Stagflation (เงินเฟ้อสูง + หุ้นตก เช่นปี 2022) ทองคำจะทำหน้าที่เป็น Alpha Generator ให้พอร์ต
3. **Universe แนะนำสำหรับ Production:** ["SPY", "QQQ", "DIA", "TLT", "GLD"] การกระจายตัวแบบข้ามสินทรัพย์ (Cross-Asset) จะทำให้ Efficient Frontier ของพอร์ตถูกยกสูงขึ้นอย่างมีนัยสำคัญ

เสาหลักที่ 4: ขกระดับ Feature Engineering (Cross-Sectional & Regime Signals)

พีเจอร์ใน V2 มีเพียง Indicator เดี่ยวของแต่ละตัว (Technical Features ทัวไป) ซึ่งไม่ได้บอก Agent ว่า "ตัวไหนแข็งแกร่งกว่าตัวไหน" ต้องเพิ่มพีเจอร์เชิงเปรียบเทียบและสภาวะตลาด:

1. **Cross-Sectional Relative Strength:**
 - $\frac{\text{Ratio Momentum: } \frac{\text{Price}_{\text{QQQ}}}{\text{Price}_{\text{SPY}}}}{\frac{\text{Price}_{\text{SPY}}}{\text{Price}_{\text{TLT}}}}$ และ $\frac{\text{Price}_{\text{SPY}}}{\text{Price}_{\text{TLT}}}$
 - Relative RSI: $\frac{\text{RSI}(\text{Asset}_i) - \text{Mean}(\text{RSI}_{\text{all}})}{\text{RSI}_{\text{all}}}$
2. **Market Regime & Macro Pressure Signals:**
 - **Yield Curve Slope:** ส่วนต่างอัตราผลตอบแทนพันธบัตร (10-Year Minus 2-Year / 3-Month Treasury Spread) เพื่อจับสัญญาณปลายวัฏจักรเศรษฐกิจ (Inversion / Disinversion)
 - **Vol of Vol & VIX Term Structure:** สัดส่วนความผันผวนระยะสั้นต่อระยะยาว ($\frac{\text{VIX}}{\text{VIX3M}}$) หรือ 5-day Realized Vol / 21-day Realized Vol)
 - **Net Liquidity Index:** สภาพคล่องสุทธิของเฟด ($\text{Fed Balance Sheet} - \text{TGA} - \text{Reverse Repo}$) ซึ่งชี้แนะกระแสเงินทุนในตลาดหุ้นสหรัฐฯ
 -

เสาหลักที่ 5: สถาปัตยกรรมโมเดลและ Action Space (PPO-LSTM + Dirichlet)

1. **เปลี่ยนจาก MLP เป็น Recurrent Policy (PPO-LSTM / GRU):**
 - พฤติกรรมของตลาดการเงินเป็น **Non-Markovian Process** (ข้อมูล $\text{Close}(t)$ เพียงวันเดียวไม่สามารถบอกสภาวะทั้งหมดได้)
 - การนำ **LSTM Policy** (MlpLstmPolicy จาก sb3-contrib) มาใช้ จะช่วยให้โมเดลมี Temporal Memory สามารถจดจำ Regime ตลาดต่อเนื่องหลายสัปดาห์ได้โดยไม่สับสน
2. **แก้ปัญหา Action Saturation ด้วย Dirichlet Distribution:**
 - V2 ใช้ Gaussian Box $[-5, 5]$ แล้วครอบด้วย Softmax ซึ่งมีปัญหา Logit ขนขอบ (Gradient หาย / ร่วงช้า)
 - ให้ปรับ Action ให้ Predict สัดส่วนความเข้มข้น (Concentration parameters α) ของ **Dirichlet Distribution** บน Simplex ($\sum w_i = 1, w_i \geq 0$) เพื่อให้หน้าหนักพอร์ตเคลื่อนที่อย่างสมูทและไม่มีค่าติดลบทางคณิตศาสตร์

ตัวอย่างการ Implement ปรับโค้ด V2 สู่ Production Core

นี่คือโค้ดส่วนของ Environment Logic ใหม่ ที่นำหลักการ Two-Tier (Tier 1: Alpha Weights + Tier 2: Vol Target Overlay) และ Alpha-Relative Reward ไปผนวกเข้ากับโครงสร้าง Zero-Leakage ของ V2:

Python

```
import numpy as np
```

```
class ProductionPortfolioEnv(gym.Env):
```

```
    """
```

Production-Grade Dual-Tier Allocation Environment:

- Tier 1: Agent selects relative risky weights across [SPY, QQQ, DIA, TLT, GLD]
- Tier 2: Realized Volatility Targeting controls Exposure vs Cash automatically
- Reward: Excess Alpha over Equal-Weight Benchmark + Downside Semi-Variance Penalty

```
    """
```

```
    def __init__(self, df, tickers, target_vol=0.10, vol_window=21):
```

```
        # tickers = ("DIA", "QQQ", "SPY", "TLT", "GLD") -> 5 สินทรัพย์หลัก
```

```
        self.tickers = tuple(tickers)
```

```
        self.k = len(self.tickers)
```

```
        self.target_vol_daily = target_vol / np.sqrt(252) # แปลง Annual Vol เป็น Daily
```

```
        self.vol_window = vol_window
```

```
        # Action space: สัดส่วนระหว่างสินทรัพย์เสี่ยงเท่านั้น (ไม่มี Cash ใน Action)
```

```
        self.action_space = spaces.Box(low=-3.0, high=3.0, shape=(self.k,), dtype=np.float32)
```

```
        # ... (Setup matrices เหมือนใน V2)
```

```
    def _get_realized_volatility(self, day_idx):
```

```
        """คำนวณ Realized Volatility ย้อนหลังของ Benchmark หรือ Assets (Zero Leakage)"""
```

```
        start_idx = max(0, day_idx - self.vol_window + 1)
```

```
        # ใช้ราคา Close ในอดีตถึง day_idx (วันตัดสินใจ t)
```

```
        closes = self.close_matrix[start_idx : day_idx + 1]
```

```
        returns = np.diff(closes, axis=0) / closes[:-1]
```

```
        # Volatility รวมเฉลี่ยของตะกร้าสินทรัพย์
```

```
        basket_returns = np.mean(returns, axis=1)
```

```
        realized_vol = np.std(basket_returns, ddof=1) if len(basket_returns) > 2 else
```

```
self.target_vol_daily
```

```
        return max(realized_vol, 1e-4)
```

```
    def step(self, actions):
```

```
        # 1. Tier 1: แปลง Action เป็น Relative Risky Weights (Sum = 1.0)
```

```
        raw_risky_weights = self._softmax(actions) # shape: (k,)
```

```
        # 2. Tier 2: Deterministic Volatility Targeting Overlay
```

```
        current_vol = self._get_realized_volatility(self.current_day)
```

```
        exposure_scalar = np.clip(self.target_vol_daily / current_vol, 0.20, 1.0)
```

```

# ผสม Risky Weights กับ Tactical Cash ตามความผันผวนจริง
final_risky_weights = raw_risky_weights * exposure_scalar
cash_weight = 1.0 - float(np.sum(final_risky_weights))

# รวมเป็น Target Allocation: [Cash, Assets...]
target_weights = np.concatenate([[cash_weight], final_risky_weights])

# 3. No-Trade Band Filter (เหมือน V2 เพื่อกัน Churn Cost)
prospective_turnover = float(np.sum(np.abs(target_weights[1:] - self.weights[1:])))
if prospective_turnover < CFG.min_turnover_threshold:
    target_weights = self.weights.copy()

# 4. Accounting Transition (Open-to-Open Zero Leakage ตาม V2)
new_val, val_weights, step_return, turnover, cost = self._transition(target_weights)

# คำนวณผลตอบแทน Benchmark ก้าวเดียวกัน (Equal-Weight Open-to-Open)
exec_open = self.open_matrix[self.current_day + 1]
next_open = self.open_matrix[self.current_day + 2]
bm_return = float(np.mean(next_open / exec_open) - 1.0)

# 5. Production Alpha Reward Formulation
excess_return = step_return - bm_return # Alpha ก้าวต่อก้าว
downside_penalty = np.square(min(0.0, step_return)) * 100.0 # ลงโทษเฉพาะขาดแรง

# Reward คือการทำผลตอบแทนชนะตลาด โดยไม่แกว่งลงรุนแรง
reward = float((excess_return * 100.0) - (downside_penalty * 0.5))

# อัปเดตสถานะพอร์ต
self.portfolio_value = new_val
self.weights = val_weights
self.current_day += 1

# ... (Bookkeeping, Obs Weights at Close(t+1) เหมือน V2)
return obs, reward, terminated, truncated, info

```

แผนงานทดสอบก่อน Deploy จริง (Production Gateways)

ก่อนนำโมเดลไปต่อ Broker API และรันเงินจริง ต้องผ่านเกณฑ์ประเมิน 4 ขั้นนี้:

1. **Deflated Sharpe Ratio (DSR > 0.95):** คำนวณ DSR (ตามระเบียบวิธีของ Marcos López de Prado) เพื่อยืนยันว่า Sharpe ที่ได้ไม่ได้มาจากการ Overfitting จากการทดลองหลายรอบ
2. **Combinatorial Purged Cross-Validation (CPCV):** ทำ Stress-test สลับลำดับข้อมูลในอดีต (Path Dependency Test) พอร์ตต้องไม่เกิด Drawdown ลึกผิดปกติในเส้นทางจำลอง
3. **Slippage & Impact Cost Stress Test:** เพิ่มสมมติฐาน Transaction Cost จาก 15 bps เป็น 30 bps (รวม Bid-Ask Spread และ Slippage ยามตลาดผันผวน) โมเดลต้องยังคงมีผลตอบแทนชนะ Benchmark
4. **Paper Trading Verification (3 เดือน):** รันสัญญาณ Forward-test ผ่าน Paper Account ทุกเข้า-ออก Open Order เพื่อตรวจสอบว่า Execution จริงไม่มี Latency และ Fill Price ตรงกับแบบจำลอง 100%